

Relatório de soluções

Contest 06/03

Henrique Simões

Março 2020

1 D - Snacktower

1.1 Ideia de solução

Primeiramente, note que podemos dizer (como saída do problema) o tamanho de um ou mais *snack(s)* a serem colocados na pilha P somente quando o maior de todos eles que ainda estavam no ar cair. Chamaremos esse maior *snack* ainda no ar como *base*. Como os tamanhos são únicos e sequenciais, ou seja, são de 1 até n , sabemos a priori qual será a primeira base: justamente o n .

Assim, podemos manter um conjunto C de quais foram os *snacks* que caíram antes da base b chegar ao chão. Com isso, certamente b será o próximo número a ser exibido — ou seja, já pode ser inserido em P — e deverá ser seguido iterativamente de todos os *snacks* $s \in C$ tais que $\exists u \in P ; s = u - 1$, sendo esses removidos de C e inseridos em P , até que $C = \emptyset$ ou $\neg \exists u \in P ; s = u - 1$, ou seja, $b - 1, b - 2, b - 3, \dots$ desde que esses tenham caído antes de b . Note que, após esse procedimento, a próxima base será necessariamente o *snack* seguinte ao último que foi inserido em P antes da condição falhar. Com a nova base determinada, basta repetir o procedimento supracitado, utilizando os mesmos conjuntos da etapa que acabou de terminar.

Note que, seguindo esse procedimento, a saída será formada conforme o especificado no enunciado do problema, com *snacks* maiores nunca em cima de menores e sempre sendo colocados assim que possível na pilha (saída padrão).

1.2 Detalhes de implementação

Tendo em vista que os números são sequenciais e são até n , que é no máximo 10^5 , podemos utilizar um vetor de identificador V para o estado de cada *snack* $i \in [1, n] \subset \mathbb{N}$. Por exemplo, 0 como sendo o *snack* ainda no ar (por vir no vetor), 1 para os que já caíram antes da base e 2 para aqueles já na pilha. Além disso, podemos manter uma variável indicando qual a próxima base b . Assim, ao encontrar b no vetor de entrada, podemos percorrer de forma decrescente nos índices o vetor V partindo da posição b enquanto a posição $b - 1$ existir e $V(b - 1) = 1$ e, simultaneamente, atualizando o valor para 2. Dessa forma, o algoritmo fica linear no número de *snacks*, ou seja, tem complexidade de tempo $\mathcal{O}(n)$.

Alternativamente, como comentado na Seção 1.1, pode-se utilizar conjuntos de fato para representar os estados na memória. Em C++, pode-se utilizar o `set<int>` e seus métodos `rbegin` e `rend` para realizar a iteração para a saída sequencial dos *snacks*. A complexidade nesse caso fica $\mathcal{O}(n \log n)$.

2 H - Three Integers

Relembre que este problema consiste na tarefa de encontrar A , B e C , tais que

1. $A \leq B \leq C$;
2. $A \mid B$ e $B \mid C$ (onde $x \mid y$ significa x divide y); e
3. $f(A, B, C) = |A - a| + |B - b| + |C - c|$ é mínimo, onde a, b, c são constantes inteiras positivas dadas na entrada do problema, satisfazendo $a \leq b \leq c$.

2.1 Ideia de solução

Uma maneira simples de solucionar esse problema é fazer a busca completa por todos os valores possíveis no espaço da solução e tomar como resposta a tripla tal que $f(A, B, C)$ atinge o menor valor durante a busca. Mostraremos que isso é viável em questão da complexidade da solução devido às restrições imposta nas variáveis.

Primeiramente, note que o limite superior dos valores que A , B e C podem assumir não é especificado. Assim, a priori eles podem ser maiores que n , onde n é o valor máximo que a, b, c podem atingir. Entretanto, podemos tomar o limite superior como sendo $4n$. Para mostrar isso, assumamos $C > 4n$. Como $c \leq n$, temos que $f(A, B, C) > 4n - n = 3n$. Por outro lado, o valor máximo plausível de f atingir como solução é menor que $n + n + n = 3n$. Portanto, $C \leq 4n$ e, de 1, segue que a solução estará no intervalo $I = [1, 4n]$.

Agora, note que, dado um valor para $A \in I$, o escopo de busca de soluções que satisfazem as restrições 1 e 2 se torna muito mais reduzido. Isso porque, nessa situação, B somente pode assumir valores tais que $A \mid B$, ou seja, valores múltiplos de A . Sabendo que existem, aproximadamente, $4n/A$ múltiplos de A em I e que calcularemos f para todo $A \in I$, segue que na ordem de

$$\sum_{i=1}^{4n} \frac{4n}{i} = 4n \sum_{i=1}^{4n} \frac{1}{i} \approx {}^1 4n \log(4n), \quad (1)$$

operações serão executadas considerando somente A e B . Como $B \mid C$, temos que uma análise análoga pode ser seguida para aproximar a ordem de operações realizadas no total para encontrar a solução do problema:

$$\sum_{i=1}^{4n} \sum_{\substack{j=ki \\ k \in \mathbb{N}^*}}^{4n} \frac{4n}{j} = \sum_{i=1}^{4n} \sum_{j=1}^{4n/i} \frac{4n}{i \cdot j} \approx 4n \log(4n) \log(4n), \quad (2)$$

que é $\mathcal{O}(n \log^2 n)$.

2.2 Detalhes de implementação

Para implementar essa solução, basta utilizar três *loops* encaixados, em que o primeiro itera sobre A em I , o segundo itera sobre B no intervalo $[A, 4n]$ com passo de A , o terceiro iterando sobre C no intervalo $[B, 4n]$ com passo de B , sendo avaliada f para cada tripla e selecionando aquela que produz o menor valor para f .

¹Veja aproximação da série de números harmônicos em <https://en.wikipedia.org/wiki/Harmonic_number> para mais informações.